Übung - Praktikum ASC

Aufgabe 1

In dieser Aufgabe vervollständigen Sie die Generierung eines deterministischen endlichen Automaten zur Umsetzung eines Scanners für die Programmiersprache TiEL. Dafür implementieren Sie die Teilmengenkonstruktion (aka Potenzmengenalgorithmus), um einen NEA in einen DEA umzuwandeln.

(a) Testen Sie Ihre Implementierung mit der zur Verfügung gestellten TiEL Scannerspezifikation. Die Implementierung ist korrekt, wenn sich der daraus erzeugte Scanner identisch zum vorgegebenen handgeschriebenen Scanner verhält. Beachten Sie auch die Implementierungshinweise unten.

(b) Bearbeiten Sie die Aufgabe in einer Gruppe von maximal 2 Personen. Die Gruppenzusammenstellung ist verbindlich auch für die Folgeaufgaben.

(c) Lassen Sie Ihr Projekt bis inkl. **18.11.2025** von den Betreuungspersonen abnehmen, dabei müssen alle Gruppenmitglieder anwesend sein und den Code erklären können.

Bewertungskriterien: Ihr Code muss lauffähig sein, die Teilmengenkonstruktion muss korrekt implementiert sein UND Sie müssen in der Lage sein, den Transfer von der Theorie zu Ihrer Implementierung zu erklären. **Eine korrekte Implementierung alleine ist nicht ausreichend**.

Implementierungshinweise

- Diese und folgende Aufgaben werden innerhalb des TiEL-Interpreters implementiert. Falls noch nicht geschehen, laden Sie sich das Projekt aus dem Moodle-Kurs herunter und richten Sie es in Ihrer Entwicklungsumgebung (z.B. IntelliJ) ein.
- Implementieren Sie die fehlenden Methoden `epsilonClosure` und `constructWithStateMap` der `de.thm.asc.generators.scanning.automata.PowersetConstruction` Klasse.
- Die zugrundeliegende Scannerspezifikation finden Sie in der Klasse `TiELScannerGenerated`. Aus dieser Spezifikation wird zunächst mittels Thompson-Konstruktion ein NEA erzeugt. Dieser wird dann mittels Ihrer Implementierung der Teilmengenkonstruktion in einen DEA umgewandelt. Es lohnt sich, den Ablauf dieser Transformationen im Code nachzuvollziehen.
- Für die Erzeugung des Automaten können Sie die Klasse `FiniteAutomatonBuilder` verwenden, die Ihnen Methoden zum Hinzufügen von Zuständen und Übergängen bereitstellt.
- Mit der Kommadozeilenoption `--scanner=GENERATED` wechseln sie vom handgeschriebenen Scanner zum generierten Scanner, der auf dem erzeugten DEA basiert.
- Mit der Kommadozeilenoption `--tokens` können Sie alle erkannten Tokens ausgeben lassen.
- Mit den Kommandozeilenoptionen `--scannernfa` und `--scannerdfa` können Sie sich den erzeugten NEA bzw. DEA als Graph dot-Code ausgeben lassen. Dot-Code können Sie mithilfe von Graphviz visualisieren, (z.B. unter `https://dreampuf.github.io/GraphvizOnline`).
- Es steht Ihnen ein JUnit-Test `TiELScannerGeneratedTest` zur Verfügung, der Ihren generierten Scanner mit dem vorgegebenen handgeschriebenen Scanner vergleicht. Wenn beide Scanner für alle Testprogramme identische Tokenfolgen erzeugen, ist Ihr generierter Scanner vermutlich korrekt implementiert. Die beigelegten Testprogramme erheben jedoch keinen Anspruch auf Vollständigkeit, Sie sollten Ihre Implementierung auch mit eigenen Testfällen überprüfen.

Aufgabe 2

In dieser Aufgabe vervollständigen Sie die Implementierung eines LL(1)-Parsers für TiEL. Dafür implementieren Sie die Berechnung der First- und Follow-Mengen sowie die Erstellung der LL(1)-Parsertabelle. Außerdem entwickeln Sie eine LL(1)-Grammatik für TiEL.

(a) Machen Sie sich mit den vorgegebenen Klassen im Package `de.thm.asc.generators.parsing` vertraut.

(b) Implementieren Sie Algorithmen zur Berechnung der First- und Follow-Mengen. Für Ihre Implementierung gibt es in den Klassen `FirstSets` und `FollowSets` bereits entsprechende Platzhalter.

(c) Implementieren Sie die Erstellung der LL(1)-Parsertabelle. Dazu müssen Sie in der Klasse `LL1Table` die statische Methode `constructForGrammar()` implementieren.

(d) Erstellen Sie in der Klasse `TiELParserGenerated` eine LL(1)-Grammatik für TiEL. Denken Sie auch an die semantischen Aktionen, die aus dem konkreten Parsebaum einen abstrakten Syntaxbaum konstruieren.

(e) Lassen Sie Ihr Projekt bis inkl. **16.12.2025** von den Betreuungspersonen abnehmen. Bewertungskritierien siehe oben.

Implementierungshinweise

- Es steht Ihnen eine Beispielgrammatik in der Klasse `ParserExample` zur Verfügung. Orientieren Sie sich an diesem Beispiel, um zu lernen, wie mit den vorgegebenen Methoden Grammatiken definiert werden können.
- Mit der Kommandozeilenoption `--ll1table` können Sie sich die errechneten First- und Follow-Mengen sowie die Parsertabelle ausgeben lassen.
- Mit der Kommandozeilenoption `--parser=GENERATED` wechseln Sie vom handgeschriebenen Parser zum generierten LL(1)-Parser.
- Mit der Kommandozeilenoption `--ast` können Sie sich den abstrakten Syntaxbaum (AST) des geparsten TiEL-Programms ausgeben lassen.
- Mit der Kommandozeilenoption `--ll1trace` können Sie die Ausführung des LL(1)-Parsers nachverfolgen. Es wird angezeigt, welche Regeln angewendet werden und wie sich der Parser-Stack verändert.
- Der JUnit-Test `ParserExampleTest` testet die Korrektheit der LL1-Algorithmen anhand der Beispielgrammatik. Nutzen Sie dies, um die Korrektheit Ihrer Implementierung unabhängig von Ihrer TiEL-Grammatik zu überprüfen.
- Der Test `TiELParserTest` testet, dass der LL1-basierte Parser für TiEL identische abstrakte Syntaxbäume wie der handgeschriebene Parser erzeugt. Nutzen Sie diesen Test, um die Korrektheit Ihrer gesamten Implementierung zu überprüfen.

Aufgabe 3

Ein wesentlicher Schritt bei der Code-Erzeugung in jedem Compiler ist die Implementierung der Auswertung von Ausdrücken. Dazu wird eine Stackmaschine benutzt, die die Argumente einer Berechnung auf einen Stack pusht, Rechenoperationen anwendet und das Ergebnis wieder auf dem Stack ablegt. Diese Stackmaschine (Tiny Educational Intermediate Representation - TEIR) besitzt folgende Befehle:

- **`pushc`** <Value>: Legt eine Konstante auf dem Stack ab
- **`pushvar`** <Name>: Holt den Wert einer Variablen und legt ihn auf dem Stack ab
- **`add`**, **`mul`**, **`sub`**, **`div`**: Entfernt die beiden obersten Stackelemente und wendet auf diese die Rechenoperation an. Das Ergebnis wird wieder auf dem Stack abgelegt.
- **`lt`**, **`gt`**, **`le`**, **`ge`**, **`eq`**, **`ne`**: Wenn der Vergleich der obersten beiden Stackelemente mit dem entsprechenden Operator true ergibt, wird true auf dem Stack abgelegt, sonst false.
- **`setvar`** <Name>: Speichert das oberste Stackelement in einer Variablen und löscht es vom Stack.
- **`pop`**: Löscht den obersten Wert auf dem Stack
- **`dup`**: Dupliziert den obersten Wert
- **`call`**: Springt zu der Funktion, deren Name oben auf dem Stack liegt; die zu übergebenden Argumente müssen ebenfalls auf dem Stack liegen. Nach Ende des Aufrufs liegt das Ergebnis wieder auf dem Stack.

Beispiel: Der Ausdruck a+f(x)*3 wird übersetzt in die TEIR–Anweisungen:

```
pushvar a
pushvar x
pushc f
call
pushc 3
mul
add

```

Beispiel: Der Ausdruck x > (a=3) wird übersetzt in die TEIR–Anweisungen:

```
pushvar x
pushc 3
dup
setvar a
ge

```

Die Klasse `de.thm.asc.teir.StackInstruction` stellt die Befehle als Java Records zur Verfügung.

Ihre Aufgabe:

Vervollständigen Sie in der Klasse `de.thm.asc.tiel.interpreter.backend.codegeneration.CodeGenerator` die Methode `generate`, um einen TiEL-Ausdruck in TEIR-Anweisungen zu übersetzen. Die virtuelle Maschine, um TEIR auszuführen, ist vorgegeben (`de.thm.asc.teir.TeIRVirtualMachine`).

Lassen Sie Ihre Lösung bis inkl. **10.2.2026** von den Betreuungspersonen abnehmen.

Aufgabe 4

Bonusaufgabe (6 Punkte):

(a) Erweitern Sie Ihren Compiler, sodass vollständige TiEL-Programme in TEIR übersetzt werden. Dafür stehen folgende neue Instruktionen zur Verfügung:

- Label: Definiert ein Sprungziel mit dem Namen Label.
- **jmp** <Label>: Setzt die Programmausführung bei dem Label fort.
- **jmpiftruthy** <Label>: Setzt die Programmausführung bei dem Label fort, wenn das oberste Stackelement true ist.
- **jmpiffalsy** <Label>: Setzt die Programmausführung bei dem Label fort, wenn das oberste Stackelement false ist.
- **openframe**: Öffnet einen neuen Stackframe
- **closeframe**: Schließt den aktuellen Stackframe
- **return**: Kehrt wieder zur Aufrufstelle zurück
- **createvar** <Name>: Erzeugt Speicher für eine Variable auf dem aktuellen Stackframe
- **createclosure** <Label>: Erzeugt eine Closure auf dem Stack

Vor einem Funktionsaufruf müssen die Argumente der Funktion von links nach rechts auf den Stack gepusht werden. Nach der Ausführung der Call-Instruktion steht das Ergebnis des Aufrufs auf dem Stack.

Beispiel: Eine Funktion

```
fun addiere(a,b) {
    return a+b;
}
```

Wird übersetzt in:

```
addiere:
  openframe
  createvar a
  setvar a
```

```
createvar b
setvar b
pushvar a
pushvar b
add
closeframe
return
```

(b) Lassen Sie Ihre Lösung bis inkl. **10.2.2026** von den Betreuungspersonen abnehmen. Sollte Ihre Lösung nicht vollständig funktionieren, können Sie Teilpunkte erhalten. Die Bonuspunkte werden direkt auf Ihr Klausurergebnis angerechnet.